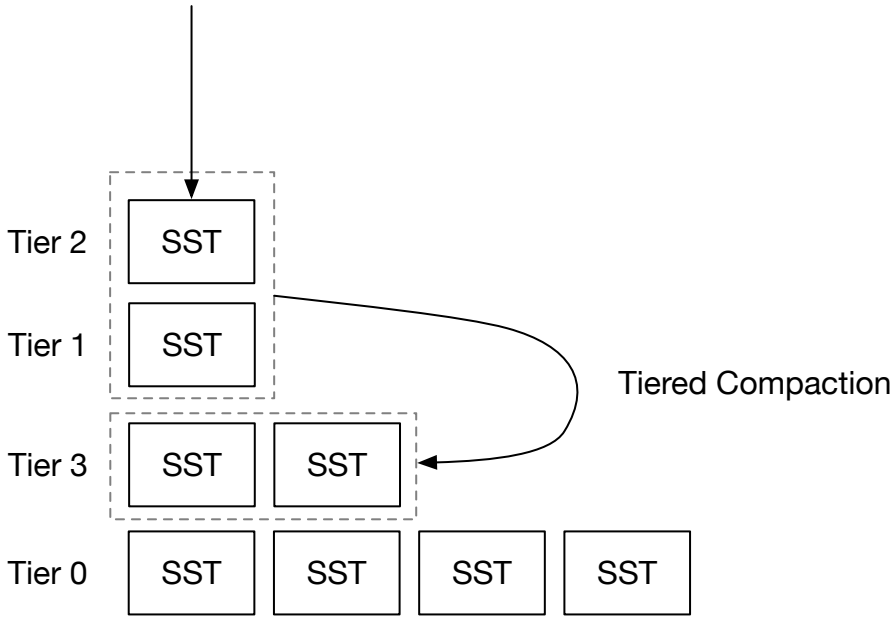
# Tiered Compaction Strategy




In this chapter, you will:

- Implement a tiered compaction strategy and simulate it on the compaction simulator.
- Incorporate tiered compaction strategy into the system.

The tiered compaction we talk about in this chapter is the same as RocksDB's universal compaction. We will use these two terminologies interchangeably.

To copy the test cases into the starter code and run them,

```
cargo x copy-test --week 2 --day 3
cargo x scheck
```

> It might be helpful to take a look at week 2 overview before reading this chapter to have a general overview of compactions.

## Task 1: Universal Compaction

In this chapter, you will implement RocksDB's universal compaction, which is of the tiered compaction family compaction strategies. Similar to the simple leveled compaction strategy, we only use number of files as the indicator in this compaction strategy. And when we trigger the compaction jobs, we always include a full sorted run (tier) in the compaction job.

## Task 1.0: Precondition

In this task, you will need to modify:

```
src/compact/tiered.rs
```

In universal compaction, we do not use L0 SSTs in the LSM state. Instead, we directly flush new SSTs to a single sorted run (called tier). In the LSM state, `levels` will now include all tiers, where **the lowest index is the latest SST flushed**. Each element in the `levels` vector stores a tuple: level ID (used as tier ID) and the SSTs in that level. Every time you flush L0 SSTs, you should flush the SST into a tier placed at the front of the vector. The compaction simulator generates tier id based on the first SST id, and you should do the same in your implementation.

Universal compaction will only trigger tasks when the number of tiers (sorted runs) reaches `num_tiers`. Otherwise, it does not trigger any compaction.

## Task 1.1: Triggered by Space Amplification Ratio

The first trigger of universal compaction is by space amplification ratio. As we discussed in the overview chapter, space amplification can be estimated by `engine_size / last_level_size`. In our implementation, we compute the space amplification ratio by `all levels except last level size / last level size`, so that the ratio can be scaled to `[0, +inf)` instead of `[1, +inf]`. This is also consistent with the RocksDB implementation.

The reason why we compute the space amplification ratio like this is because we model the engine in a way that it stores a fixed amount of user data (i.e., assume it's 100GB), and the user keeps updating the values by writing to the engine. Therefore, eventually, all keys get pushed down to the bottom-most tier, the bottom-most tier size should be equivalent to the amount of data (100GB), the upper tiers contain updates to the data that are not yet compacted to the bottom-most tier.

When `all levels except last level size / last level size >= max_size_amplification_percent * 1%`, we will need to trigger a full compaction. For example, if we have a LSM state like:

```
Tier 3: 1
Tier 2: 1 ; all levels except last level size = 2
Tier 1: 1 ; last level size = 1, 2/1=2
```

Assume `max_size_amplification_percent` = 200, we should trigger a full compaction now.

After you implement this trigger, you can run the compaction simulator. You will see:

```
cargo run --bin compaction-simulator tiered --iterations 10
```

```
=== Iteration 2 ===
--- After Flush ---
L3 (1): [3]
L2 (1): [2]
L1 (1): [1]
--- Compaction Task ---
compaction triggered by space amplification ratio: 200
L3 [3] L2 [2] L1 [1] -> [4, 5, 6]
--- After Compaction ---
L4 (3): [3, 2, 1]
```

With this trigger, we will only trigger full compaction when it reaches the space amplification ratio. And at the end of the simulation, you will see:

```
cargo run --bin compaction-simulator tiered
```

```
=== Iteration 7 ===
--- After Flush ---
L8 (1): [8]
L7 (1): [7]
L6 (1): [6]
L5 (1): [5]
L4 (1): [4]
L3 (1): [3]
L2 (1): [2]
L1 (1): [1]
--- Compaction Task ---
--- Compaction Task ---
compaction triggered by space amplification ratio: 700
L8 [8] L7 [7] L6 [6] L5 [5] L4 [4] L3 [3] L2 [2] L1 [1] -> [9, 10, 11, 12, 13,
14, 15, 16]
--- After Compaction ---
L9 (8): [8, 7, 6, 5, 4, 3, 2, 1]
--- Compaction Task ---
1 compaction triggered in this iteration
--- Statistics ---
Write Amplification: 16/8=2.000x
Maximum Space Usage: 16/8=2.000x
Read Amplification: 1x

=== Iteration 49 ===
--- After Flush ---
L82 (1): [82]
L81 (1): [81]
L80 (1): [80]
L79 (1): [79]
L78 (1): [78]
L77 (1): [77]
L76 (1): [76]
L75 (1): [75]
L74 (1): [74]
L73 (1): [73]
L72 (1): [72]
L71 (1): [71]
L70 (1): [70]
L69 (1): [69]
L68 (1): [68]
L67 (1): [67]
L66 (1): [66]
L65 (1): [65]
L64 (1): [64]
L63 (1): [63]
L62 (1): [62]
L61 (1): [61]
L60 (1): [60]
L59 (1): [59]
L58 (1): [58]
L57 (1): [57]
L33 (24): [32, 31, 30, 29, 28, 27, 26, 25, 24, 23, 22, 21, 20, 19, 18, 17, 9,
10, 11, 12, 13, 14, 15, 16]
--- Compaction Task ---
--- Compaction Task ---
no compaction triggered
```

```
--- Statistics ---
Write Amplification: 82/50=1.640x
Maximum Space Usage: 50/50=1.000x
Read Amplification: 27x
```

The `num_tiers` in the compaction simulator is set to 8. However, there are far more than 8 tiers in the LSM state, which incurs large read amplification.

The current trigger only reduces space amplification. We will need to add new triggers to the compaction algorithm to reduce read amplification.

## Task 1.2: Triggered by Size Ratio

The next trigger is the size ratio trigger. The trigger maintains the size ratio between the tiers. From the first tier, we compute the size of `this tier / sum of all previous tiers`. For the first encountered tier where this value `> (100 + size_ratio) * 1%`, we will compact all previous tiers excluding the current tier. We only do this compaction with there are more than `min_merge_width` tiers to be merged.

For example, given the following LSM state, and assume `size_ratio` = 1, and `min_merge_width` = 2. We should compact when the ratio value > 101%:

```
Tier 3: 1
Tier 2: 1 ; 1 / 1 = 1
Tier 1: 1 ; 1 / (1 + 1) = 0.5, no compaction triggered
```

Example 2:

```
Tier 3: 1
Tier 2: 1 ; 1 / 1 = 1
Tier 1: 3 ; 3 / (1 + 1) = 1.5, compact tier 2+3


Tier 4: 2
Tier 1: 3
```

Example 3:

```
Tier 3: 1
Tier 2: 2 ; 2 / 1 = 2, however, it does not make sense to compact only one
tier; also note that min_merge_width=2
Tier 1: 4 ; 4 / 3 = 1.33, compact tier 2+3


Tier 4: 3
Tier 1: 4
```

With this trigger, you will observe the following in the compaction simulator:

```
cargo run --bin compaction-simulator tiered
```

```
=== Iteration 49 ===
--- After Flush ---
L119 (1): [119]
L118 (1): [118]
L114 (4): [113, 112, 111, 110]
L105 (5): [104, 103, 102, 101, 100]
L94 (6): [93, 92, 91, 90, 89, 88]
L81 (7): [80, 79, 78, 77, 76, 75, 74]
L48 (26): [47, 46, 45, 44, 43, 37, 38, 39, 40, 41, 42, 24, 25, 26, 27, 28, 29,
30, 9, 10, 11, 12, 13, 14, 15, 16]
--- Compaction Task ---
--- Compaction Task ---
no compaction triggered
--- Statistics ---
Write Amplification: 119/50=2.380x
Maximum Space Usage: 52/50=1.040x
Read Amplification: 7x
```

```
cargo run --bin compaction-simulator tiered --iterations 200 --size-only
```

```
=== Iteration 199 ===
--- After Flush ---
Levels: 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 3 4 5 6 10
15 21 28 78
no compaction triggered
--- Statistics ---
Write Amplification: 537/200=2.685x
Maximum Space Usage: 200/200=1.000x
Read Amplification: 38x
```

There will be fewer 1-SST tiers and the compaction algorithm will maintain the tiers to have smaller to larger sizes by size ratio. However, when there are more SSTs in the LSM state, there will still be cases that we have more than `num_tiers` tiers. To limit the number of tiers, we will need another trigger.

## Task 1.3: Reduce Sorted Runs

If none of the previous triggers produce compaction tasks, we will do a major compaction that merges SST files from the first up to `max_merge_tiers` tiers into one tier to reduce the number of tiers.

With this compaction trigger enabled, you will see:

```
cargo run --bin compaction-simulator-ref tiered --iterations 200 --size-only
```

```
=== Iteration 199 ===
--- After Flush ---
Levels: 0 1 1 4 5 21 28 140
no compaction triggered
--- Statistics ---
Write Amplification: 742/200=3.710x
Maximum Space Usage: 280/200=1.400x
Read Amplification: 7x
```

You can also try tiered compaction with more number of tiers:

```
cargo run --bin compaction-simulator tiered --iterations 200 --size-only --num-
tiers 16
```

```
=== Iteration 199 ===
--- After Flush ---
Levels: 0 1 1 1 1 1 1 1 1 1 15 175
no compaction triggered
--- Statistics ---
Write Amplification: 607/200=3.035x
Maximum Space Usage: 350/200=1.750x
Read Amplification: 12x
```

**Note: we do not provide fine-grained unit tests for this part. You can run the compaction simulator and compare with the output of the reference solution to see if your implementation is correct.**

# Task 2: Integrate with the Read Path

In this task, you will need to modify:

```
src/compact.rs
src/lsm_storage.rs
```

As tiered compaction does not use the L0 level of the LSM state, you should directly flush your memtables to a new tier instead of as an L0 SST. You can use `self.compaction_controller.flush_to_l0()` to know whether to flush to L0. You may use the first output SST id as the level/tier id for your new sorted run. You will also need to modify your compaction process to construct merge iterators for tiered compaction jobs.

# Related Readings

Universal Compaction - RocksDB Wiki

# Test Your Understanding

- What is the estimated write amplification of leveled compaction? (Okay this is hard to estimate... But what if without the last *reduce sorted run* trigger?)
- What is the estimated read amplification of leveled compaction?
- What are the pros/cons of universal compaction compared with simple leveled/tiered compaction?
- How much storage space is it required (compared with user data size) to run universal compaction?
- Can we merge two tiers that are not adjacent in the LSM state?
- What happens if compaction speed cannot keep up with the SST flushes for tiered compaction?
- What might needs to be considered if the system schedules multiple compaction tasks in parallel?
- SSDs also write its own logs (basically it is a log-structured storage). If the SSD has a write amplification of 2x, what is the end-to-end write amplification of the whole system? Related: ZNS: Avoiding the Block Interface Tax for Flash-based SSDs.
- Consider the case that the user chooses to keep a large number of sorted runs (i.e., 300) for tiered compaction. To make the read path faster, is it a good idea to keep some data structure that helps reduce the time complexity (i.e., to `O(log n)`) of finding SSTs to read in each layer for some key ranges? Note that normally, you will need to do a binary search in each sorted run to find the key ranges that you will need to read. (Check out Neon's layer map implementation!)

We do not provide reference answers to the questions, and feel free to discuss about them in the Discord community.